

Assignment #3

Function Calling

Dr. Daniel M. Yellin

DRAFT

THESE SLIDES ARE THE PROPERTY OF DANIEL YELLIN
THEY ARE ONLY FOR USE BY STUDENTS OF THE CLASS
THERE IS NO PERMISSION TO DISTRIBUTE OR POST
THESE SLIDES TO OTHERS

</motivation>

The problem

You work at mycompany.com

Employees and customer submit requests to mycompany.com. Most of these requests are answered by a dedicated team that look at the request, find the company files that contain the appropriate data, analyze the data, and return a response.

Your manager has asked you to automate this process using an LLM.

The challenge

When you examine the queries and responses the team delivers, you find it is more complicated than your manager thought:

- Delivering an answer can require multiple steps
- The files used to answer queries are textual files (.txt), spreadsheets (.csv), and pictures (.png)
- Queries can require analysis of all of these different type of files.
- Output can take multiple forms: a textual, a numeric answer, a data plot, a word cloud, or an event timeline

Your solution

You took a class in college on building agent-based systems using function calling. You realize it could possibly be a really good solution for your problem since:

- The solution will require intermixing LLM calls with Agent/Tool calls.
- Different queries will require different tools, and even if they require the same tools, they can differ in the order in which the tools must be invoked.
- Trying to write code to figure out how to cover all the cases would be hard to write and impossible to maintain.

</motivation >

Due date

Assignment is due June 24, 2025 at 23:59

24.06.25 @23:59

Assignment

Build an agent that can answer queries. Each query may require multiple steps, including reading input files, and taking actions.

- This assignment specifies the list of actions you can take. You will need to implement each action and the agent control flow to invoke the actions.
- You do not know in advance what the query will be, which specific actions will be required, or how many steps (different actions) it will take. You only know that it is solvable using the tools and resources at your disposal.

Tools available

Your agent is equipped with the following tools (functions):

1. An LLM, which is used for determining which tool to use at each step. It can also be used by each individual tool as needed.
 - For example, the Internet Search tool will call the LLM after retrieving search results from the Internet so that the LLM can extract the answer from the search results and format it into JSON.
2. A tool to extract entities from a text file
3. An Internet Search Tool
 - This tool will take very specific types of queries and return specific types of answers
4. A tool to write a Python program to analyze csv files and to generate a plot (using Matplotlib)
5. A tool to run a Python program
6. A tool to debug and regenerate a faulty Python program
7. A tool to write information to a file

The exact requirements of each tool are given later in this presentation

Input to your program is a file “input.json” of the following form

```
{
  "query-name": "<query_name>.txt",
  "resources": [
    {
      "name": "<file_name>",
      "description": "<some_string>"
    },
    {
      "name": "<file_name>",
      "description": "<some_string>"
    }
    ...
  ]
}
```

```
{
  "query_name": "query1.txt",
  "file_resources": [
    {
      "file_name": "grades.csv",
      "description": "A csv file describing students and their
grades. the first line contains the columns (field names) of each
following row. They are: First, Last, Year, Class, Name, Grade.
An example row is: Shalom,Levi,3,Calculus,93.5"
    },
    {
      "file_name": "students.txt",
      "description": "a text file containing information on some of
the students, including his or her aspirations, favorite city, and
hobbies."
    }
  ]
}
```

input.json

Name of the query file. It can be any name and might include a path to a different directory.

Name of a file resource the solution can use.

```
{
  "query-name": "<query_name>.txt",
  "resources": [
    {
      "name": "<file_name>",
      "description": "<some_string>"
    },
    {
      "name": "<file_name>",
      "description": "<some_string>"
    },
    ...
  ]
}
```

```
{
  "query_name": "query1.txt",
  "file_resources": [
    {
      "file_name": "grades.csv",
      "description": "A csv file describing students and their grades. the first line contains the columns (field names) of each following row. They are: First, Last, Year, Class, Name, Grade. An example row is: Shalom,Levi,3,Calculus,93.5"
    },
    {
      "file_name": "students.txt",
      "description": "a text file containing information on some of the students, including his or her aspirations, favorite city, and hobbies."
    }
  ]
}
```

Examples

For these examples, two resource files are defined, `grades.csv` and `students.txt`

grades.csv

First, Last, University-Year, Class, Grade

Shalom, Levi, 3, Calculus, 93

Nitzi, Matok, 1, Intro to programming, 89

Chava, Bina, 3, Algorithms, 77

Maor, Katz, 2, Automaton, 69

Hinon Chalek, 3, Calculus, 82

Rutie, Levana, 2, Algorithms, 95

...

students.txt

Shalom Levi wants to become a doctor. His favorite city is Tel Aviv and his hobby is playing chess.

Nitzi Matok loves Eilat. Her hobby is scuba diving.

Chava Bina hopes to become an entrepenuer. She would like to live in Haifa. For a hobby, she does crossword puzzles.

Example: query1.txt

Find the hobbies of all students. The result should be JSON array where each object in the array has one field, 'hobby', whose value is the name of the hobby. Store the JSON in the file "query1.json".

Output file query1.json:

```
[{"hobby": "playing chess"}, {"hobby": "scuba diving"}, {"hobby": "crossword puzzles"}]
```

Example: query2.txt

Find the current population of a city that is mentioned in the students.txt file that begins with the letter 'H'. Write the result as a JSON object with the fields 'city' and 'population', whose value is a number. The JSON should be stored in the file "query2.json".

Agent log

calling chat completions

** Entering Agent extract_entities_from_file **

** Leaving Agent extract_entities_from_file **

calling chat completions

** Entering Agent Internet_search_attribute **

** Leaving Agent Internet_search **

calling chat completions

** Entering Agent write_file **

** Exiting Agent write_file **

calling chat completions

final response is = The user's query was successfully solved. The answer can be found in the file "query2.json", which contains the JSON object with the city "Haifa" and its population of 290,306.

Process finished with exit

For the assignment, instead of “calling chat completions” it should say “Calling LLM for next tool to invoke”

You can use the terminology “Agent” or “Tool”. The latter is more consistent with the terminology we have been using.

query2.json

```
{"city": "Haifa", "population": 290306}
```

World-happiness-report-2024.csv

Country name,Regional indicator,Ladder score,upperwhisker,lowerwhisker,Log GDP per capita,Social support,Healthy life expectancy,Freedom to make life choices,Generosity,Perceptions of corruption,Dystopia + residual

Finland,Western Europe	7.741,7.815,7.667,1.844,1.572,0.695,0.859,0.142,0.546,2.082
Denmark,Western Europe	7.583,7.665,7.500,1.908,1.520,0.699,0.823,0.204,0.548,1.881
Iceland,Western Europe	7.525,7.618,7.433,1.881,1.617,0.718,0.819,0.258,0.182,2.050
Sweden,Western Europe	7.344,7.422,7.267,1.878,1.501,0.724,0.838,0.221,0.524,1.658
Israel,Middle East and North Africa	7.341,7.405,7.277,1.803,1.513,0.740,0.641,0.153,0.193,2.298
Netherlands,Western Europe	7.319,7.383,7.256,1.901,1.462,0.706,0.725,0.247,0.372,1.906
Norway,Western Europe	7.302,7.389,7.215,1.952,1.517,0.704,0.835,0.224,0.484,1.586
Luxembourg,Western Europe	7.122,7.213,7.031,2.141,1.355,0.708,0.801,0.146,0.432,1.540
Switzerland,Western Europe	7.060,7.147,6.973,1.970,1.425,0.747,0.759,0.173,0.498,1.488
Australia,North America and ANZ	7.057,7.141,6.973,1.854,1.461,0.692,0.756,0.225,0.323,1.745
New Zealand,North America and ANZ	7.029,7.105,6.954,1.810,1.527,0.673,0.746,0.226,0.480,1.567
Costa Rica,Latin America and Caribbean	6.955,7.051,6.860,1.561,1.373,0.661,0.797,0.109,0.123,2.333
Kuwait,Middle East and North Africa	6.951,7.060,6.843,1.845,1.364,0.661,0.827,0.200,0.172,1.884
Austria,Western Europe	6.905,6.986,6.824,1.885,1.336,0.696,0.703,0.214,0.305,1.766
Canada,North America and ANZ	6.900,6.984,6.815,1.840,1.459,0.701,0.730,0.230,0.368,1.572
Belgium,Western Europe	6.894,6.961,6.827,1.868,1.440,0.690,0.729,0.170,0.311,1.686
Ireland,Western Europe	6.838,6.927,6.749,2.129,1.390,0.700,0.758,0.205,0.418,1.239

...

NOTE: the data is normalized. To understand the numbers see <https://worldhappiness.report/ed/2024/>

Column-explanation.txt

Country name: Name of the country.

Regional indicator: Region to which the country belongs.

Ladder score: The happiness score for each country, based on responses to the Cantril Ladder question that asks respondents to think of a ladder, with the best possible life for them being a 10, and the worst possible life being a 0.

Upper whisker: Upper bound of the happiness score.

Lower whisker: Lower bound of the happiness score.

Log GDP per capita: The natural logarithm of the country's GDP per capita, adjusted for purchasing power parity (PPP) to account for differences in the cost of living between countries.

Social support: The national average of binary responses (either 0 or 1 representing No/Yes) to the question about having relatives or friends to count on in times of trouble.

Healthy life expectancy: The average number of years a newborn infant would live in good health, based on mortality rates and life expectancy at different ages.

Freedom to make life choices: The national average of responses to the question about satisfaction with freedom to choose what to do with one's life.

Generosity: The residual of regressing the national average of responses to the question about donating money to charity on GDP per capita.

Perceptions of corruption: The national average of survey responses to questions about the perceived extent of corruption in the government and businesses.

Dystopia + residual: Dystopia is an imaginary country with the world's least-happy people, used as a benchmark for comparison. The dystopia + residual score is a combination of the Dystopia score and the unexplained residual for each country, ensuring that the combined score is always positive. Each of these factors contributes to the overall happiness score, but the Dystopia + residual value is a benchmark that ensures no country has a lower score than the hypothetical Dystopia.

Positive affect: The national average of responses to questions about positive emotions experienced yesterday.

Negative affect: The national average of responses to questions about negative emotions experienced yesterday.

input.json and WHR.txt

input.json

```
{
  "query_name": "query1/WHR.txt",
  "file_resources": [
    {
      "file_name": "query1/World-happiness-report-2024.csv",
      "description": "A csv file describing world happiness scores countries. the first line contains the columns (field names) of each following row. They are: 'Country name','Regional indicator','Ladder score','upperwhisker','lowerwhisker','Log GDP per capita', 'Social support','Healthy life expectancy','Freedom to make life choices','Generosity','Perceptions of corruption', 'Dystopia'"
    }
  ]
}
```

WHR.txt

Create a program to plot the average happiness score and the happiness scores of the top 5 and the lowest 5 countries in the file query1/World-happiness-report-2024.csv. It should also plot the average happiness score of all countries in that file. The program should store the .png plot in a file and then terminate. Execute the program to generate the plot.

Agent log

calling chat completions

** Entering Agent gen_plot_prog **

** Exiting Agent gen_plot_prog **

calling chat completions

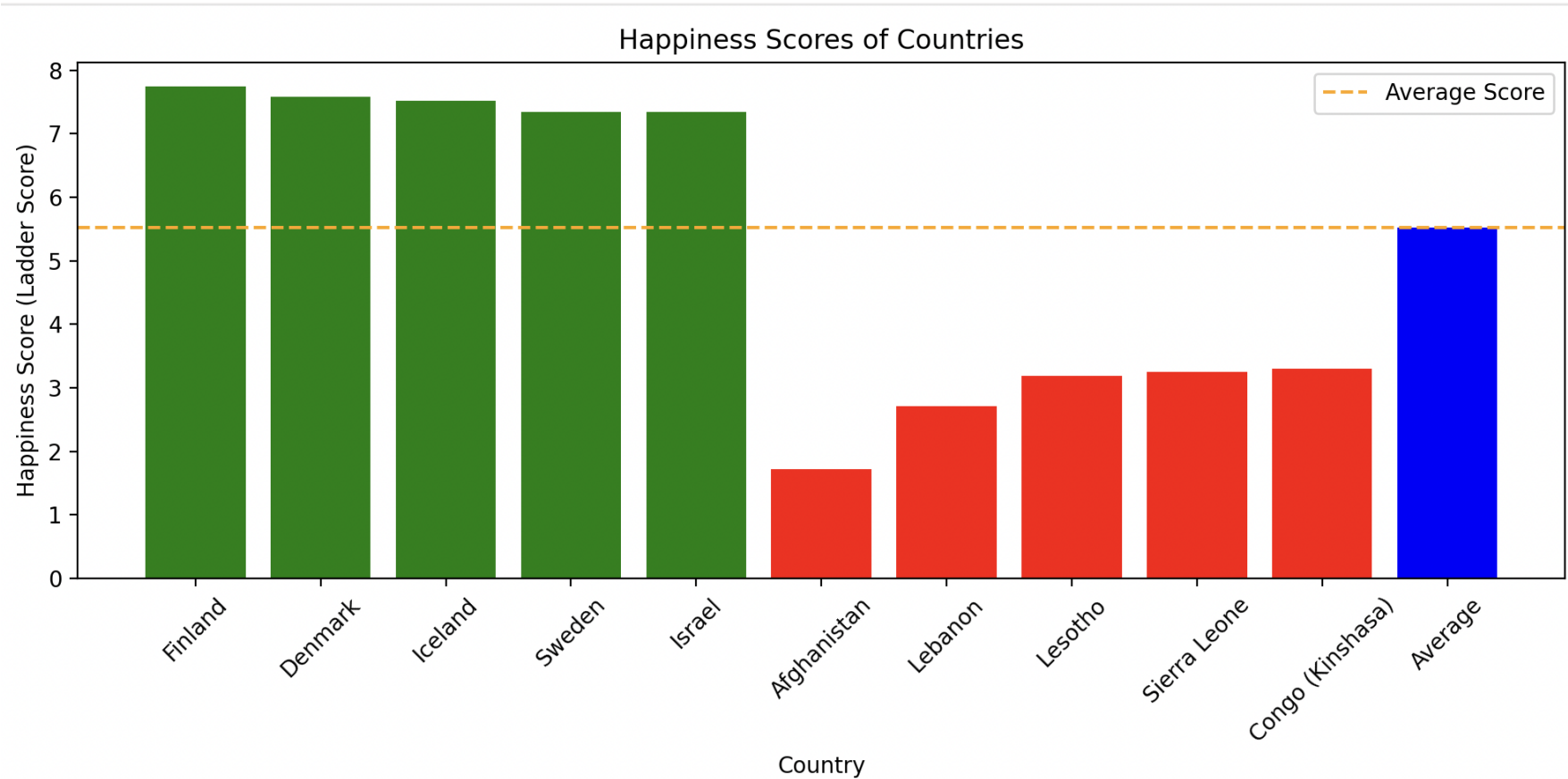
** Entering Agent execute_Python_program **

** Exiting Agent execute_Python_program **

calling chat completions

final response is = The user's query has been successfully solved. The program to plot the average happiness score and the happiness scores of the top 5 and lowest 5 countries has been executed, and the resulting plot is saved as "happiness_scores_plot.png".

Plot generated by program (happiness_scores_plot.png)



A second example

input.json

```
• {  
  "query_name": "query2/WHR2.txt",  
  "file_resources": [  
    {  
      "file_name": "query1/World-happiness-report-2024.csv",  
      "description": "A csv file describing world happiness scores countries. the first line contains the columns (field names) of  
each following row. They are: 'Country name','Regional indicator','Ladder score','upperwhisker','lowerwhisker','Log GDP per  
capita','Social support','Healthy life expectancy','Freedom to make life choices','Generosity','Perceptions of  
corruption','Dystopia'"  
    }  
  ]  
}
```

WHR2.txt

Find a country that belongs to the UN but is NOT listed in the file query1/World-happiness-report-2024.csv.

Store the result in the file "query2/result.json" as a JSON structure with one field:
"name_of_country".

Agent log

calling chat completions

** Entering Agent extract_entities_from_file **

** Exiting Agent extract_entities_from_file **

calling chat completions

** Entering Agent write_file **

** Exiting Agent write_file **

calling chat completions

final response is = The query was successfully solved. You can find the answer in the file "query2/result.json", which contains the name of a country that belongs to the UN but is not listed in the World Happiness Report 2024.

result.json

```
{"name_of_country": "Andorra"}
```

Yes, it really is a country <https://en.wikipedia.org/wiki/Andorra> and it is not part of the UN!

On another run I got the answer Vatican City, which is not a correct answer. It is a country (the smallest in the world) but “Vatican City is not a member state of the United Nations. However, it maintains a permanent observer mission at the UN.”

I also got Afghanistan a couple of time, which is incorrect, as it is listed in the [World-happiness-report-2024.csv](#) file.

A third example

input.json

```
{
  "query_name": "query3/WHR3.txt",
  "file_resources": [
    {
      "file_name": "query1/World-happiness-report-2024.csv",
      "description": "A csv file describing the world happiness scores for many countries. the first line contains the columns (field names) of each following row. They are: 'Country name','Regional indicator','Ladder score','upperwhisker','lowerwhisker','Log GDP per capita','Social support','Healthy life expectancy','Freedom to make life choices','Generosity','Perceptions of corruption','Dystopia'"
    },
    {
      "file_name": "query3/Best-countries-to-raise-families.txt",
      "description": "This file discusses the top 7 countries in which to raise families."
    }
  ]
}
```

WHR3.txt

Find those countries rated as one of the top 7 countries to raise families using the appropriate file resource. We'll call them the 7C countries. Store the 7C countries in "query3/top-7.json" where the json contains one field, "top-7", an array giving the names of the 7 countries.

Next, generate a program, using the appropriate file resource, to plot the average "Healthy life expectancy" for all countries (using "Country name" column), the average "Healthy life expectancy" for the 7C countries, and the "Healthy life expectancy" for the country that has the highest "Healthy life expectancy" score. In the plot state the name of the country that has the highest "Healthy life expectancy" score, and the values of the averages. Execute the program and generate the plot.

Agent log

calling chat completions

** Entering Agent extract_entities_from_file **

** Exiting Agent extract_entities_from_file **

calling chat completions

** Entering Agent write_file **

** Exiting Agent write_file **

** Entering Agent gen_plot_prog **

** Exiting Agent gen_plot_prog **

calling chat completions

** Entering Agent execute_Python_program **

** Exiting Agent execute_Python_program **

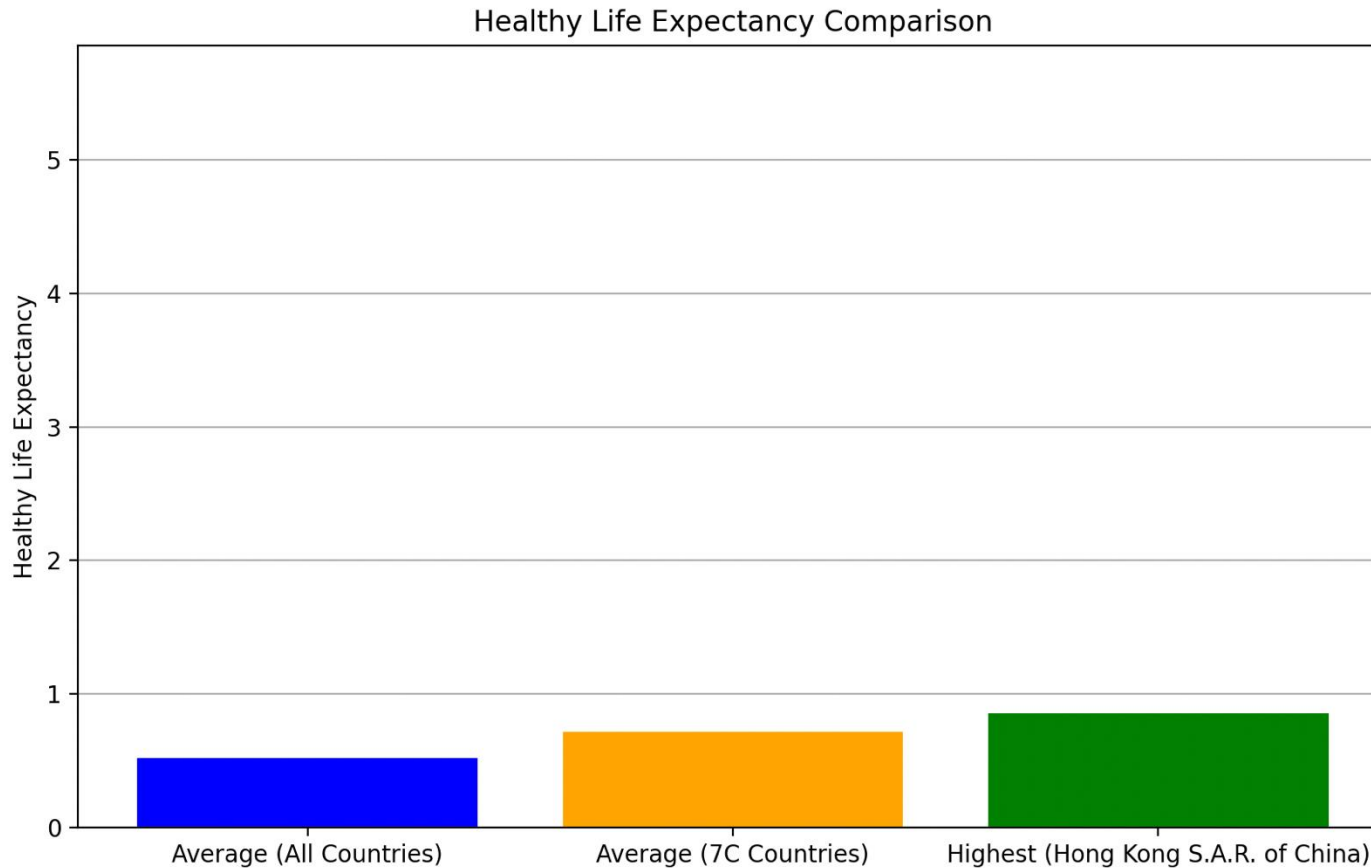
calling chat completions

final response is = The user's query has been successfully solved. The countries rated as the top 7 countries to raise families can be found in the file "query3/top-7.json". Additionally, the plot comparing the average "Healthy life expectancy" for all countries, the average for the 7C countries, and the highest scoring country can be found as a PNG image in "query1/health_life_expectancy_plot.png".

top-7.json

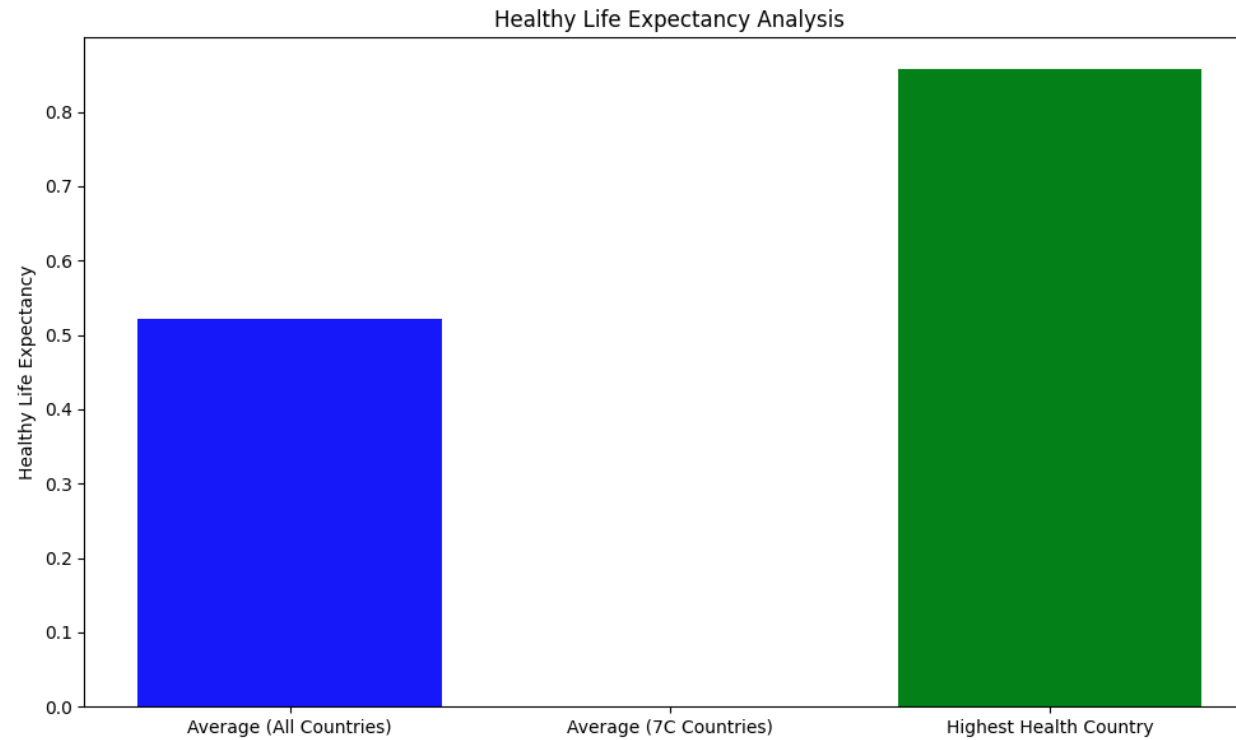
```
{"top-7": ["Iceland", "Denmark", "Sweden", "Finland", "Norway", "Netherlands", "Switzerland"]}
```

One incorrect plot generated

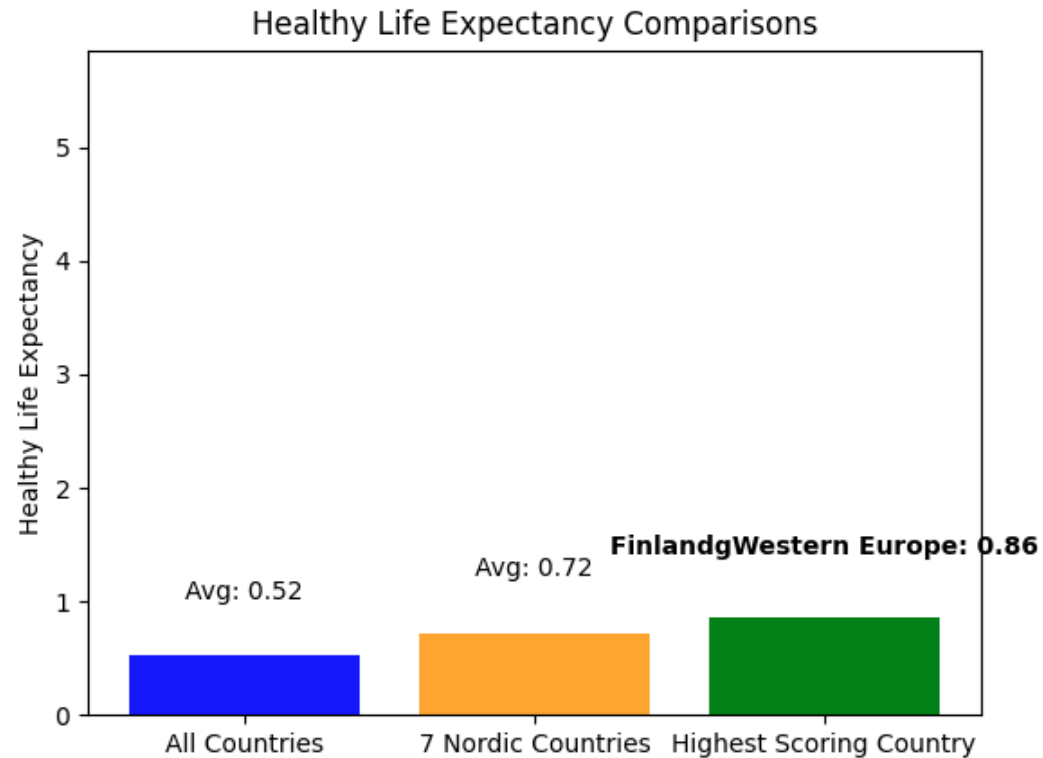


Hong Kong is .857

Another incorrect plot



This time it almost gets it right



Should be 7C countries

Finland is .859 (the highest)

Assignment guidelines

Functions (tools) available for your program to use

1. **extract_entities_from_file(file_name: str, entity_type: str) -> str**
This tool is given the name of a file and an “entity type”, a string. It returns a comma-separated list of all the entities of that type in the file. For example, if the file content is “I lived in Chicago and Milwaukee” and the entity type is “city”, it would return [‘Chicago’, ‘Milwaukee’]. The list it returns is a string of the form “[entity1, entity2,...,entityk]”.
2. **Internet_search_attribute(an_entity: str, an_attribute: str) -> str:**
This tool searches the Internet to find the desired attribute of the given entity. For example, if the entity is "Abraham Lincoln", and the attribute is "birthdate", it would search the Internet to find the birthdate of Abraham Lincoln. Similarly, if the entity is "Canada", and the attribute is "current prime minister", it would search the Internet for the current prime minister of Canada. After receiving the search results, it then asks the LLM to review the results and return a JSON structure of the form: {"entity": an_entity, "attribute": an_attribute, "attribute_value": <value>}, where <value> is the value extracted from the Internet search results.

Notes on Internet_search_attribute tool

You can use any search engine you want. Use one that gives you free queries. I use DuckDuckGo (see `Duckduckgo_search` Python package). Alternatively you can use SerpAPI (<https://serpapi.com>) or another of your choice or take one from LangChain.

When you issue a request to fetch results from the Internet, you can set how many results you want to return. You should just return 3 results.

Actions (tools) available to your program

3. **gen_plot_prog(plot_request: str, input_file: str, columns: str, gen_output_program_fn:str, output_png: str):**

This tool receives a `plot_request` (a string) which describes a query to be performed on the `input_file` (a csv file). This `input_file` has the names of the columns in its first line, and every following line is a row of data. The names of the columns are also supplied as an input parameter to this agent. The tool produces a program `P` that computes the `plot_request` and stores the generated plot in the .png file `output_png`. The tool then writes the generated program `P` to the file `gen_output_file`. It returns the generated program.

Notes on gen_plot_prog tool

I had a lot of trouble with this tool because it would not only generate the plot and save it in a file, it would also issue a command to display the plot, and then would not terminate until I terminated the program.

To fix this issue, I added to my prompt the following:

“...The generated program should store the graph in the .png file {output_png} using the command plt.savefig. The program must NOT use the plt.show command, as then the program will not terminate.”
(This is not the entire prompt). Feel free to use this text or some text of your own.

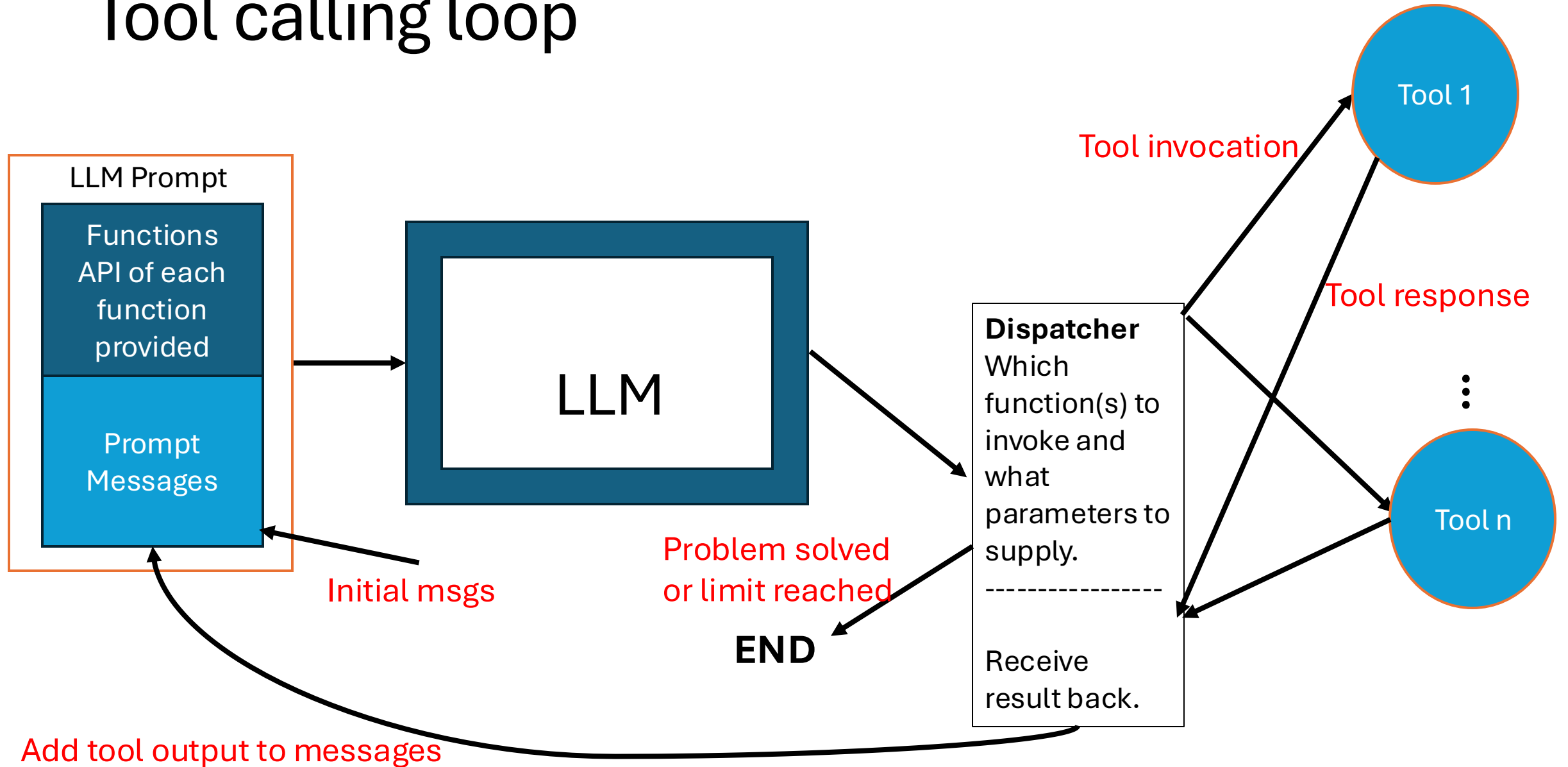
Actions (tools) available to your program

4. **execute_Python_prog(program_fn:str) -> str:**
This tool executes the program in file program_fn. If the program executes successfully, it returns the string 'Program executed successfully'. Otherwise, it returns the error messages from stderr.
5. **debug_and_regenerate_prog(program_fn:str, errors: str) -> str:**
This tool takes the name of a file which contains a Python program. This program executed unsuccessfully generating the given errors. The tool first reflects on the errors to find the root cause. It then generates a new program that fixes those errors. It stores the newly generated program in the same file, overwriting the old program.
6. **write_file(file_content:str, fn:str)**
This tool writes the file contents to the file fn.

Agent rules

- Your agent must use these tools with the same names. However, you can change the parameters of these functions if you desire.
- You can add a couple more tools if you want (but not necessary).
 - For instance, you can have an LLM check the answer to a question (LLM as a judge).
 - Or you can have a tool that removes `plt.show` if it appears in a plot program.
 - Neither of these are necessary!
- All your program can do is to prompt the LLM or to invoke one of the functions.
- The program cannot prompt the LLM more than 10 times, and cannot invoke tools (all together) more than 10 times. If a tool invokes an LLM, that counts for one of the LLM 10 invocations.

Tool calling loop



Tool calling loop

The following pseudocode is more or less how your program should work. Of course, there are lots of details to fill in.

```
messages = [initial_system_message,
            initial_user_message]
while not done and not at limit:
    response =
client.chat.completions.create(messages,...)
    if response is recommending tool T and parameters
p1,...,pk
        tool_result = T(p1,...,pk)
        messages.append(tool_result_message)
summarize final status using LLM.
```

Getting good results

To get good results, you will need to formulate the the prompts to the LLM. Better prompts will get better results.

You also need to provide a good clear description of each tool and its parameters, so the LLM will know which tool to use.

In general, document clearly but succinctly. If the description is too long, it can confuse the LLM. If they are under-specified, you will often get incorrect answers.

Required outputs

What your program must output

1. The query will define what outputs are expected.
2. If the solution requires generation of a Python program, or a json file, the query will define what file to store it in.
3. Logging tool entry and exit
 - Every time you enter tool A, you need to log “**Entering tool A**”.
 - You should log, for each parameter p of A, “Parmeter p = <value of p>”. **If the parameter value is a string, just record the first 50 chars** of the string.
 - Every time you exit tool A, you need to log “**Exiting tool A**”.
 - The log should be saved in query_name.log, where query_name is given in the input.json file (without the “.txt” extension).

Submission

- You should submit your application and any other files required to run your application on Moodle.
- You must include a file “Partners” that lists all the students working on this submission, even if it is a single person.

LLM and environment variables

You should use the 'gpt-4o-mini' model

- MODEL_4o = 'gpt-4o-mini'
- OPENAI_API_VERSION_4o = '2024-08-01-preview'

As in Assignments 1 and 2, you need to initialize LLM variables from the environment:

```
AZURE_OPENAI_API_KEY = os.getenv('CLASS_OPENAI_API_KEY')
```

```
AZURE_OPENAI_ENDPOINT_4o =  
    os.getenv('SUBSCRIPTION_OPENAI_ENDPOINT_4o')
```

The value of these variables are the same as in those assignment 2.

You must use the environment variable names as given above (the ones in `os.getenv`). Those will be the environment names on the Bodek's computer.

How we will grade

- Checking that the program runs correctly and all required outputs are produced.
- Checking that logging is done properly.
- Invoking each function individually and checking that it works correctly.
- Checking that you have implemented function calling correctly.
- Producing the correct answers.
 - That is why using the correct prompt techniques and building good function and parameter descriptions as part of the JSON function schema is important.

Check before submitting

- In the last assignment, some assignments could not be run because of some “small” mistake, or because incorrect environment variables were used, ...
- We will be stricter this time on requiring the submitted program to run on the first try. Please test your program accurately on another computer before submitting.